

In [1]: `import pandas as pd`

```
df = pd.read_csv('messy_sales.csv')

print("--- Shape ---")
display(df.shape)

print("\n--- First 10 Rows ---")
display(df.head(10))

print("\n--- Column Data Types ---")
display(df.dtypes)
```

--- Shape ---

(300, 6)

--- First 10 Rows ---

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>order_id</th> <th>date</th> <th>product</th> <th>price</th> <th>qty</th>
<th>zip</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody>
<tbody></tbody>

1254	04/06/2026	mug	11.36	4	60614
------	------------	-----	-------	---	-------

1057	2026-05-30	charger	14.79	3	30303
------	------------	---------	-------	---	-------

1150	05/06/2026	mug	5.50	1	10001
------	------------	-----	------	---	-------

1066	05/30/2026	notebook	NaN	1	98101
------	------------	----------	-----	---	-------

1114	04/06/2026	webcam	45.59	4	90405
------	------------	--------	-------	---	-------

1280	04/22/2026	pen set	NaN	4	30303
------	------------	---------	-----	---	-------

1204	04/18/2026	charger	8.41	2	2134
------	------------	---------	------	---	------

1249	2026-05-15	keyboard	75.04	4	90405
------	------------	----------	-------	---	-------

1037	2026-05-19	mug	9.31	2	60614
------	------------	-----	------	---	-------

1177	2026-05-13	mug	10.11	3	90210
------	------------	-----	-------	---	-------

--- Column Data Types ---

order_id int64

date str

product str

price float64

qty int64

zip int64

dtype: object

<p>Three problems I can see with this dataset are:</p>

1. One of the zip codes is truncated because the zip column is stored as a numeric type (int64), which strips away leading zeros.
2. The dates are formatted differently.
3. Several cells are missing values (designated "NaN").

```
In [2]: import matplotlib.pyplot as plt

print("--- Missing Value Counts ---")
missing_counts = df.isna().sum()
display(missing_counts)
```

```
--- Missing Value Counts ---
order_id    0
date        0
product     0
price       12
qty         0
zip         0
dtype: int64
```

```
In [3]: fill_value = df["price"].median()

df["price"] = df["price"].fillna(fill_value)

print(f"Fill value used: {fill_value}")
print(f"Remaining missing prices in 'price': {df['price'].isna().sum()}")
```

Fill value used: 37.53

Remaining missing prices in 'price': 0

<p>I chose the median over the mean because price data tends to be skewed towards higher prices, and a few high priced items can artificially raise the mean.</p>

```
In [4]: import matplotlib.pyplot as plt

print("--- Missing Value Counts ---")
missing_counts = df.isna().sum()
display(missing_counts)
```

```
--- Missing Value Counts ---
order_id    0
date        0
product     0
price       0
qty         0
zip         0
dtype: int64
```

```
In [5]: print(f"Duplicates found: {df.duplicated().sum()}")

print(f"Shape before: {df.shape}")

df = df.drop_duplicates()

print(f"Shape after: {df.shape}")
```

Duplicates found: 8
Shape before: (300, 6)
Shape after: (292, 6)

```
In [6]: df["zip"] = df["zip"].astype(str).str.zfill(5)

print(df["zip"].head(10))
```

```
0    60614
1    30303
2    10001
3    98101
4    90405
5    30303
6    02134
7    90405
8    60614
9    90210
Name: zip, dtype: str
```

```
In [7]: df["date"] = pd.to_datetime(df["date"], format="mixed")

print(df.dtypes)
```

```
order_id      int64
date          datetime64[us]
product       str
price         float64
qty           int64
zip           str
dtype: object
```

```
In [8]: print("--- Shape Check ---")
print(df.shape)

print("\n--- Data Types Check ---")
print(df.dtypes)

print("\n--- Zip Code Check ---")
print(df[["zip"]].head())
print("Unique zip code lengths:", df["zip"].str.len().unique())
```

```
--- Shape Check ---
(292, 6)
```

```
--- Data Types Check ---
order_id          int64
date              datetime64[us]
product           str
price             float64
qty              int64
zip              str
dtype: object
```

```
--- Zip Code Check ---
      zip
0  60614
1  30303
2  10001
3  98101
4  90405
Unique zip code lengths: [5]
```

```
In [9]: print("--- Rows with Negative Quantities ---")
negative_rows = df[df["qty"] < 0]
display(negative_rows)
print(f"Total rows found: {len(negative_rows)}")

df = df[df["qty"] >= 0]

print(f"\nShape after handling negative quantities: {df.shape}")
```

```
--- Rows with Negative Quantities ---
```

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>order_id</th> <th>date</th> <th>product</th> <th>price</th> <th>qty</th>
<th>zip</th> </thead> <tbody> <th>202</th> <th>262</th> <th>297</th> </tbody>
<tbody></tbody>
```

1140	2026-04-18	webcam	38.69	-5	98101
1233	2026-05-09	desk lamp	22.64	-4	10001
1025	2026-04-27	keyboard	47.30	-2	02116

```
Total rows found: 3
```

```
Shape after handling negative quantities: (289, 6)
```

```
In [10]: # Save the cleaned DataFrame to a new CSV file without the index column
df.to_csv("sales_clean.csv", index=False)

print("File successfully saved as 'sales_clean.csv'!")
```

```
File successfully saved as 'sales_clean.csv'!
```

<h2 id="Cleaning-Log">Cleaning Log</h2>

- **Loaded Dataset:** Initial load of `messy_sales.csv` with a shape of **(300, 6)**.
- **Handled Missing Prices:** Filled missing values in the `price` column using the **median** (to resist skewed prices from outliers), ensuring 0 missing price cells remained.
- **Removed Duplicates:** Detected and dropped **8 duplicate rows**, bringing the row count down to **292**.
- **Repaired Zip Codes:** Converted the `zip` column to text strings and applied `.str.zfill(5)` to restore truncated leading zeros (ensuring proper 5-digit postal codes).
- **Standardized Dates:** Converted the mixed-format `date` column into standard `datetime64[ns]` objects using `format="mixed"`.
- **Handled Negative Quantities:** Filtered and removed rows with negative quantities (`qty < 0`), treating them as returns or data entry errors to prevent them from corrupting sales volume metrics.

<p>*Conclusion:* Making improper cleaning choices, such as filling missing prices with an inflated mean or leaving negative return rows unaddressed, can severely distort financial analytics and lead executives to make flawed inventory, pricing, and revenue-forecasting decisions.</p>

In []: